Critical Reproduction Study:
Network Intrusion Detection with Autoencoders on NSL-KDD

Course: Data Science in Cyber (Dr. Uri Itai)
Source: Network Intrusion Detection with Autoencoders - Steven Foerster
https://stevenfoerster.com/tutorials/network-intrusion-detection-with-autoencoders/
No official GitHub repo for the article. Dataset mirror: github.com/defcom17/NSL_KDD

Executive Summary

We reproduced the source protocol: PyTorch autoencoder trained on normal_train only (50 epochs), Isolation Forest with 200 trees on normal_train only, and both thresholds calibrated on the same normal_validation split at the 99th percentile. Strict preprocessing fits the anomaly scaler on normal_train only and removes constant features such as num_outbound_cmds.

At the source operating point (99th percentile), autoencoder F1=0.737, recall=0.607; Isolation Forest F1=0.718, recall=0.567. The tutorial claims a modest, dataset-dependent advantage for autoencoders over Isolation Forest - not universal superiority over all classical methods. Our threshold-sensitivity analysis shows higher recall at 95% is threshold-dependent and does not by itself prove better feature interaction learning.

1. Summary of the Source

Problem: binary/normal-vs-attack intrusion detection on NSL-KDD connection records.
Source protocol: 50 autoencoder epochs, MSE reconstruction error, 99th percentile threshold on held-out normal validation traffic, 200-tree Isolation Forest trained on the same normal_train, threshold also calibrated on normal_validation scores at the 99th percentile.
Claim: autoencoders may outperform Isolation Forest on some attack types by learning feature relationships, but the advantage is modest and dataset-dependent.

2. Critical Evaluation

Our earlier 95th-percentile result used a different precision-recall trade-off and is not the fair comparison to the source. The correct reproduction uses 99th percentile on normal_validation for both anomaly models.

Threshold choice directly controls FPR/FNR: lower percentiles increase recall but also false alarms. We do NOT claim autoencoders capture patterns tree models miss unless shown under the same operating point. Lower raw false-negative counts at 95% may reflect threshold calibration, not proven feature-interaction superiority.

At 99%: Autoencoder F1=0.737, recall=0.607; Isolation Forest F1=0.718, recall=0.567. At 95% both models achieve higher recall, but that is a different operating point. Supervised Random Forest remains competitive when labels are available.

3. Feature Engineering

Encoding fit on KDDTrain+ only. Constant columns removed: ['num_outbound_cmds']. Anomaly scaler fit on normal_train only - attack rows no longer influence anomaly preprocessing. Supervised scaler fit on all KDDTrain+ rows because labels are used. Feature ablation compares full features, constant removal, and redundant-feature removal.

4. Reproducibility

In this submission environment we re-ran:

```
pip install -r requirements.txt
python run_pipeline.py
python generate_report.py
```

The pipeline completed successfully in about 146 seconds on CPU. A fresh clone still requires downloading

KDDTrain+.txt and KDDTest+.txt into data/. Small numeric differences vs the original tutorial may occur because of package versions and random seeds, but the protocol (50 AE epochs, 200 IF trees, 99th percentile on the same normal_validation split) is matched.

5. Exploratory Data Analysis and Distribution Shift

Train attack rate: 46.5%. Test attack rate: 56.9%. This shift affects accuracy and precision/recall even when ranking quality is stable. No global timestamp exists; temporal drift analysis is not meaningful beyond connection duration.

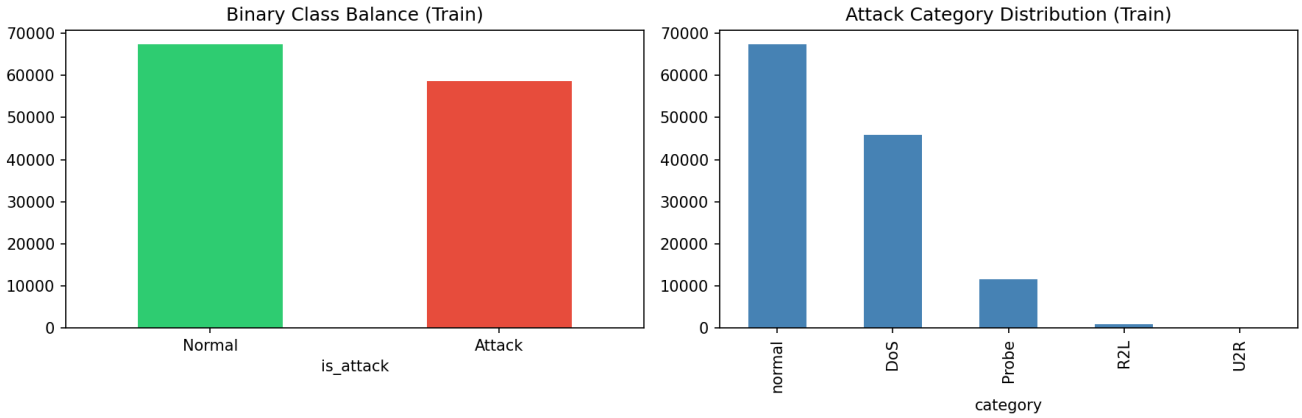

Figure 1: Class and category imbalance in KDDTrain+.

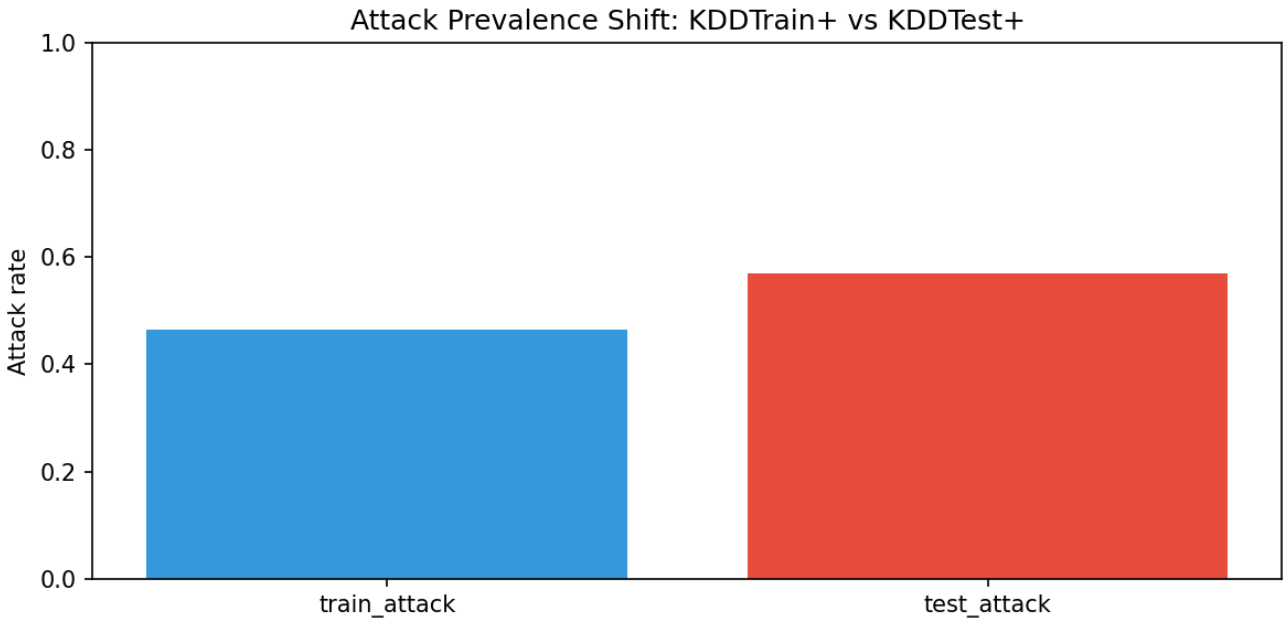

Figure 2: Train vs test attack prevalence shift.

Table 1: Distribution shift summary (excerpt).

split	rows	attack_rate	normal_rate	metric_type	category	prevalence	count	protocol_type	feature	ks_statistic	ks_pvalue	train_mean	test_mean	train_median	test_median
train	125973.0	0.465	0.535	binary_prevalence	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
test	22544.0	0.569	0.431	binary_prevalence	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
train	NaN	NaN	NaN	category_prevalence	normal	0.535	67343.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
train	NaN	NaN	NaN	category_prevalence	DoS	0.365	45927.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
train	NaN	NaN	NaN	category_prevalence	Probe	0.093	11656.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

NaN	NaN	NaN	NaN	NaN								
train	NaN	NaN	NaN	category_prevalence	R2L	0.008	995.0	NaN	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN								
train	NaN	NaN	NaN	category_prevalence	U2R	0.000	52.0	NaN	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN								
test	NaN	NaN	NaN	category_prevalence	normal	0.431	9711.0	NaN	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN								

6. Experimental Setup

normal_train rows: 53874, normal_validation rows: 13469. Supervised models trained on all KDDTrain+. Source-protocol anomaly models use normal_train / normal_validation split with 99th-percentile thresholds.

7. Source Protocol Results (99th percentile)

	accuracy	precision	recall	f1	f2	mcc	fpr	fnr	roc_auc	pr_auc
autoencoder_source_protocol	0.754	0.938	0.607	0.737	0.654	0.569	0.053	0.393	0.949	0.952
isolation_forest_source_protocol	0.747	0.980	0.567	0.718	0.619	0.581	0.016	0.433	0.942	0.955

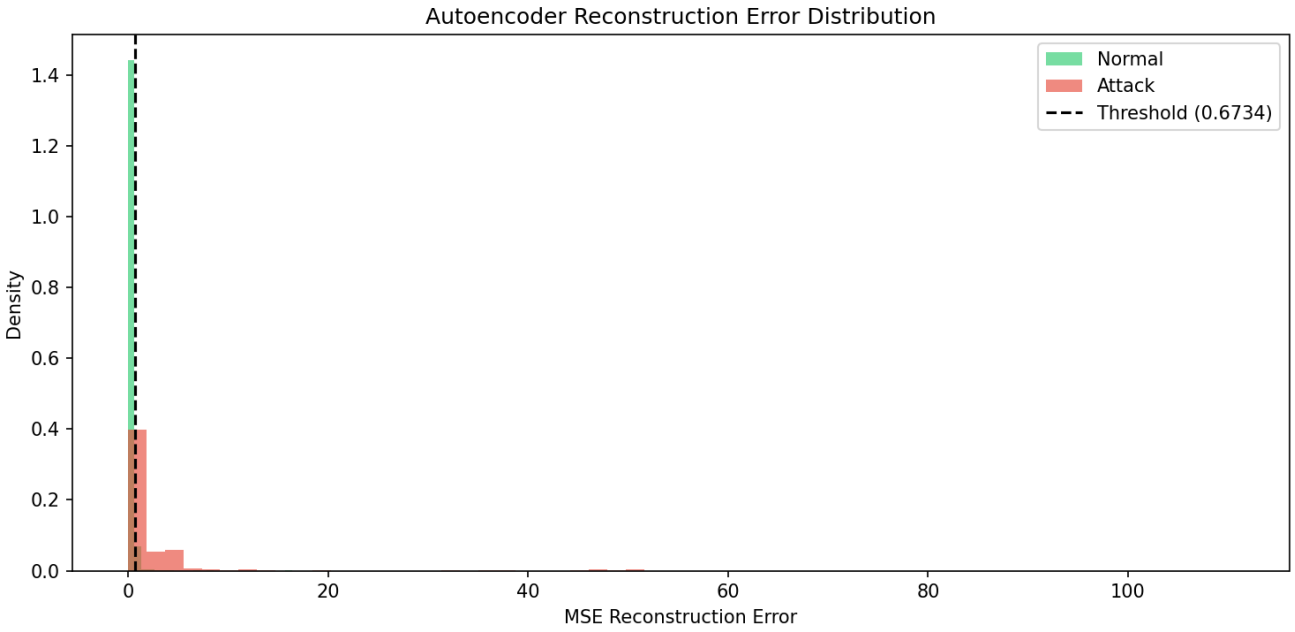

Figure 3: Reconstruction error distribution with 99th-percentile threshold.

8. Threshold Sensitivity

Both anomaly models evaluated at 95th, 97th, and 99th percentiles on normal_validation. Cyber interpretation: lower percentiles increase recall (fewer missed attacks) but raise FPR (more alert fatigue).

Table 2: Threshold sensitivity metrics.

model	percentile	threshold	accuracy	precision	recall	f1	f2	mcc	fpr	fnr	roc_auc	pr_auc
autoencoder	95	0.197	0.875	0.923	0.851	0.886	0.864	0.751	0.094	0.149	0.949	0.952
autoencoder	97	0.275	0.858	0.931	0.811	0.867	0.833	0.725	0.079	0.189	0.949	0.952
autoencoder	99	0.673	0.754	0.938	0.607	0.737	0.654	0.569	0.053	0.393	0.949	0.952
isolation_forest	95	0.486	0.801	0.965	0.676	0.795	0.719	0.650	0.033	0.324	0.942	0.955
isolation_forest	97	0.498	0.779	0.973	0.629	0.764	0.677	0.623	0.023	0.371	0.942	0.955
isolation_forest	99	0.524	0.747	0.980	0.567	0.718	0.619	0.581	0.016	0.433	0.942	0.955

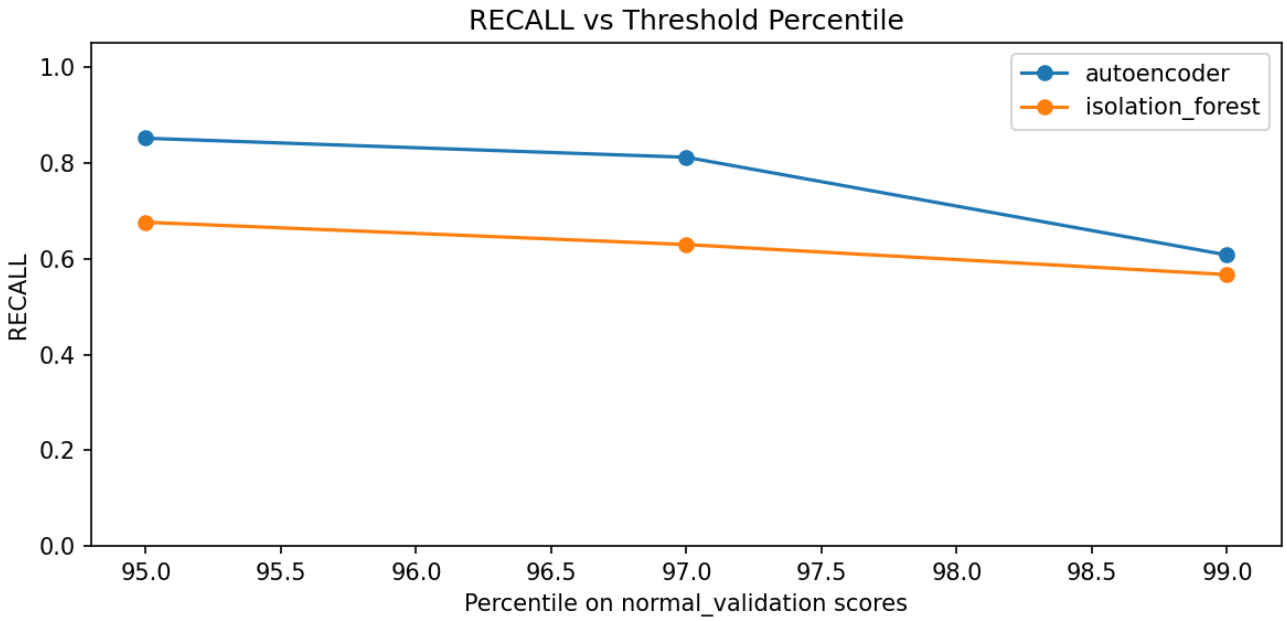

Figure 4: Recall vs threshold percentile.

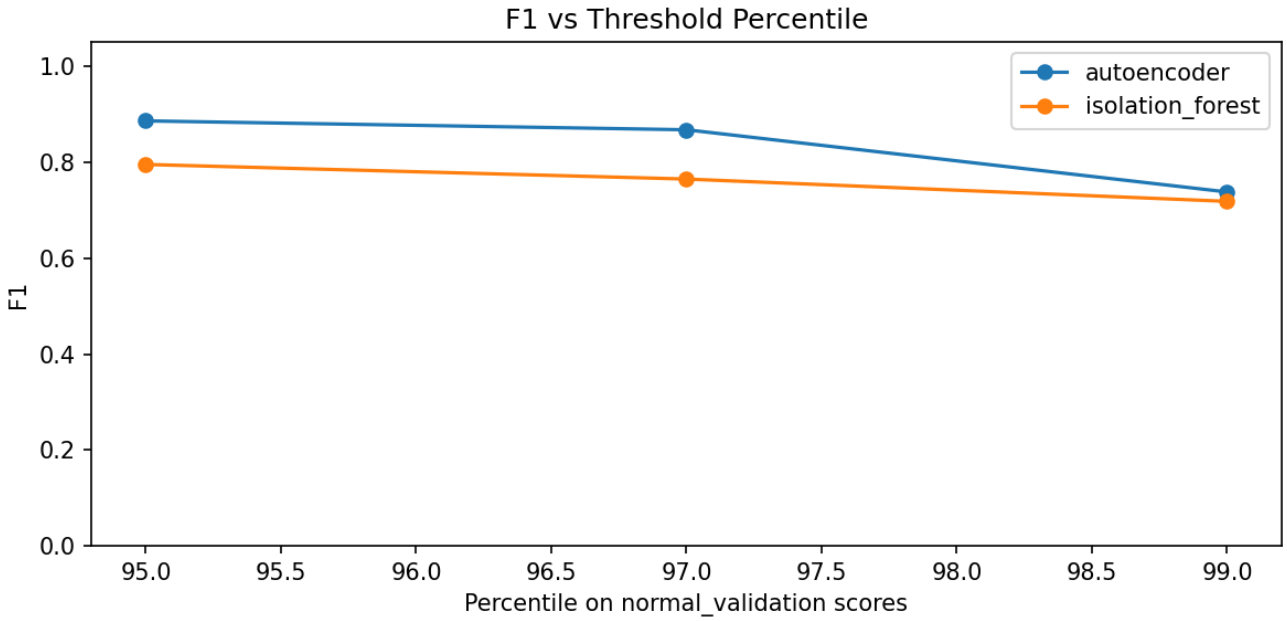

Figure 5: F1 vs threshold percentile.

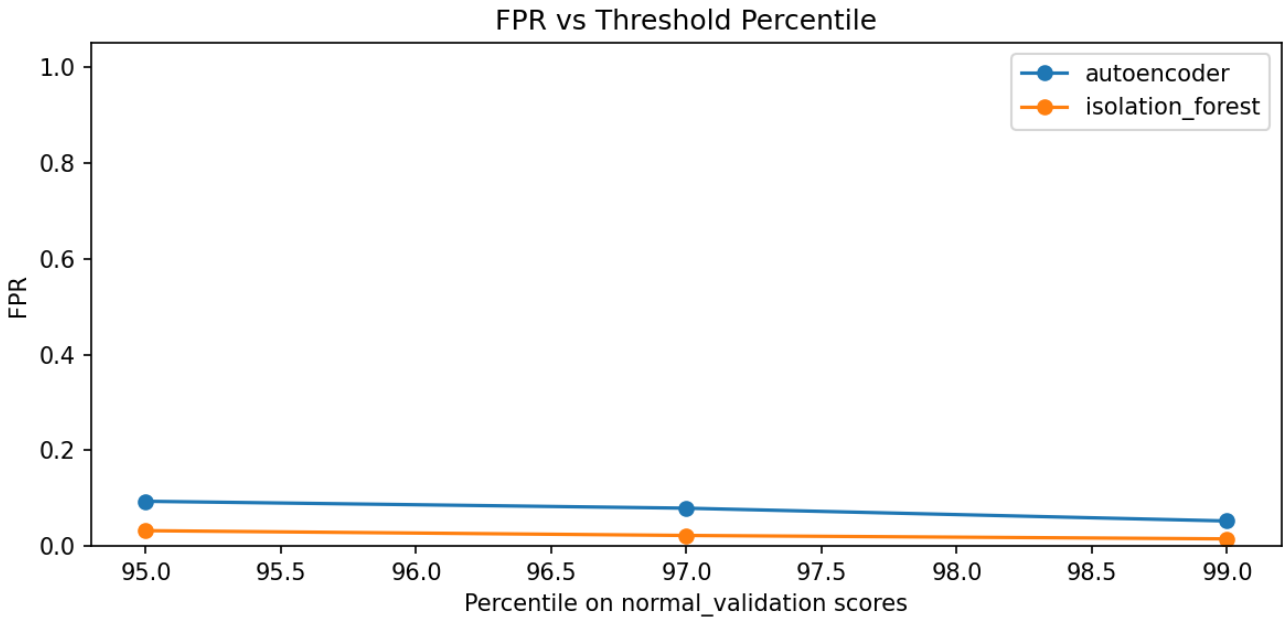

Figure 6: False positive rate vs threshold percentile.

9. Fair Anomaly Comparison

Same-percentile comparison aligns operating points. Equal-FPR comparison calibrates the autoencoder threshold on normal_validation to match the Isolation Forest FPR at 99th percentile.

Table 3: Fair anomaly comparison.

comparison_type	model	percentile	threshold	accuracy	precision	recall	f1	f2	mcc	fpr	fnr	roc_auc	pr_auc	target_fpr_on_val
same_percentile_95	autoencoder	95	0.197	0.875	0.923	0.851	0.886	0.864	0.751	0.094	0.149	0.949	0.952	NaN
same_percentile_95	isolation_forest	95	0.486	0.801	0.965	0.676	0.795	0.719	0.650	0.033	0.324	0.942	0.955	NaN
same_percentile_97	autoencoder	97	0.275	0.858	0.931	0.811	0.867	0.833	0.725	0.079	0.189	0.949	0.952	NaN
same_percentile_97	isolation_forest	97	0.498	0.779	0.973	0.629	0.764	0.677	0.623	0.023	0.371	0.942	0.955	NaN
same_percentile_99	autoencoder	99	0.673	0.754	0.938	0.607	0.737	0.654	0.569	0.053	0.393	0.949	0.952	NaN
same_percentile_99	isolation_forest	99	0.524	0.747	0.980	0.567	0.718	0.619	0.581	0.016	0.433	0.942	0.955	NaN
equal_fpr_on_normal_validation	autoencoder_equal_fpr	99	0.673	0.754	0.938	0.608	0.738	0.654	0.569	0.053	0.392	0.949	0.952	0.01
equal_fpr_on_normal_validation	isolation_forest_reference_99	99	0.524	0.747	0.980	0.567	0.718	0.619	0.581	0.016	0.433	0.942	0.955	0.01

10. Model Comparison (all models)

Table 4: Full model comparison.

Unnamed: 0	accuracy	precision	recall	f1	f2	mcc	fpr	fnr	roc_auc	pr_auc
random_forest	0.779	0.968	0.632	0.765	0.679	0.620	0.027	0.368	0.957	0.959
logistic_regression	0.752	0.916	0.622	0.741	0.665	0.556	0.075	0.378	0.805	0.874
autoencoder_source_protocol	0.754	0.938	0.607	0.737	0.654	0.569	0.053	0.393	0.949	0.952
isolation_forest_source_protocol	0.747	0.980	0.567	0.718	0.619	0.581	0.016	0.433	0.942	0.955

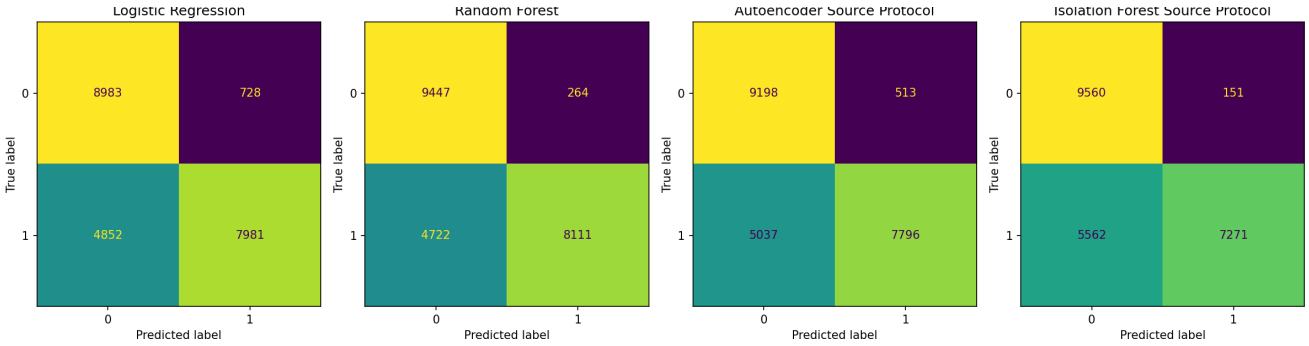

Figure 7: Confusion matrices.

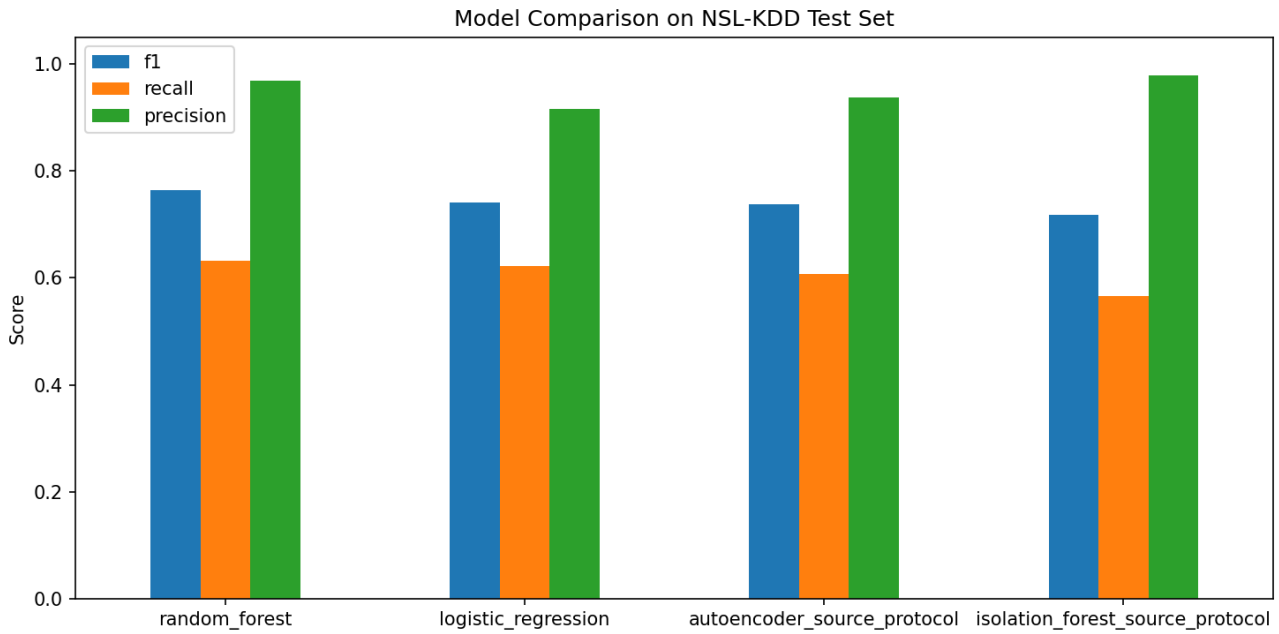

Figure 8: Metric comparison bar chart.

11. Feature Ablation

Constant features do not help and were removed in the main pipeline. Highly correlated features hurt interpretability; removing them may or may not change performance materially.

Table 5: Feature ablation results.

feature_set	num_features	model	accuracy	precision	recall	f1	f2	mcc	fpr	fnr	roc_auc	pr_auc
A_full_features	122	random_forest	0.784	0.969	0.641	0.771	0.687	0.627	0.027	0.359	0.960	0.962
A_full_features	122	autoencoder	0.748	0.986	0.565	0.719	0.618	0.586	0.010	0.435	0.964	0.964
B_remove_constants	121	random_forest	0.779	0.968	0.632	0.765	0.679	0.620	0.027	0.368	0.957	0.959
B_remove_constants	121	autoencoder	0.730	0.935	0.565	0.705	0.614	0.536	0.052	0.435	0.941	0.942
C_remove_constants_and_redundant	114	random_forest	0.780	0.969	0.635	0.767	0.682	0.622	0.027	0.365	0.959	0.960
C_remove_constants_and_redundant	114	autoencoder	0.742	0.932	0.590	0.723	0.637	0.550	0.057	0.410	0.933	0.910

12. Evaluation Metrics

Accuracy = (TP+TN)/(TP+TN+FP+FN). Precision = TP/(TP+FP). Recall = TP/(TP+FN). F1 = 2PR/(P+R). F2 = 5PR/(4P+R). MCC = (TP*TN-FP*FN)/sqrt((TP+FP)(TP+FN)(TN+FP)(TN+FN)). FPR = FP/(FP+TN). FNR = FN/(FN+TP). ROC-AUC measures ranking; PR-AUC is useful under prevalence shift. In IDS, false negatives are often more dangerous than false positives because missed intrusions persist.

13. Error Analysis - Per-Category Recall

Raw FN counts are misleading because categories have different sizes. Per-category recall = TP_in_category / total_category_samples. R2L and U2R have high security importance despite fewer samples.

Table 6: Per-category recall (excerpt).

model	category	total_samples	true_positives	false_negatives	recall
logistic_regression	DoS	7458	6102	1356	0.818
logistic_regression	Probe	2421	1843	578	0.761
logistic_regression	R2L	2754	23	2731	0.008
logistic_regression	U2R	200	13	187	0.065
random_forest	DoS	7458	6071	1387	0.814
random_forest	Probe	2421	1855	566	0.766
random_forest	R2L	2754	176	2578	0.064
random_forest	U2R	200	9	191	0.045
autoencoder_source_protocol	DoS	7458	5583	1875	0.749
autoencoder_source_protocol	Probe	2421	1894	527	0.782
autoencoder_source_protocol	R2L	2754	259	2495	0.094
autoencoder_source_protocol	U2R	200	60	140	0.300
isolation_forest_source_protocol	DoS	7458	5333	2125	0.715
isolation_forest_source_protocol	Probe	2421	1887	534	0.779
isolation_forest_source_protocol	R2L	2754	23	2731	0.008
isolation_forest_source_protocol	U2R	200	28	172	0.140

14. Conclusions

We fixed preprocessing leakage, matched the source protocol at 99th percentile, and added threshold sensitivity, fair comparison, ablation, and distribution-shift analysis. The author's claim is only partially supported: results depend on threshold and operating point. We do not overclaim feature-interaction superiority without equal-FPR evidence. Future work: modern datasets, per-attack dashboards, deployment latency.